

Fine-Tuning & Optimization, Explained

ParentWise — a plain-language and technical companion to slides 8 & 9. Each term appears twice: once for a technical audience, once for anyone.

SLIDE 8 · FINE-TUNING (QLoRA)

Teaching behaviour, not facts

“We adapted Nile-Chat-4B with QLoRA: the base model is frozen in 4-bit, and we train lightweight adapters — under one percent of the parameters — with rank 16, learning rate 1e-4, for 2 epochs. The dataset is 507 Q&A pairs (483 grounded + 24 safety/refusal), split 482 train / 25 validation / 28 held-out test with zero overlap. We fine-tune for tone and safety, not facts.”

Fine-tuning	
TECHNICAL	Continuing the training of a pre-trained language model on a smaller, task-specific dataset so its weights adapt to a target style and behaviour — instead of training a model from scratch.
IN PLAIN WORDS	Taking an AI that already speaks Arabic and giving it extra lessons so it answers like a warm, careful parenting helper. We are not building a new AI from zero — just coaching an existing one.

Nile-Chat-4B (the base model)	
TECHNICAL	An open-source, ~4-billion-parameter, Gemma-3-based large language model specialised for Egyptian Arabic. It is the starting point we adapt.
IN PLAIN WORDS	The ready-made Arabic AI we start from. “4B” means it has about 4 billion tiny internal dials it uses to produce text.

QLoRA	
TECHNICAL	Quantized Low-Rank Adaptation: a parameter-efficient fine-tuning method that loads the base model in 4-bit precision and trains only small low-rank “adapter” matrices, cutting memory use enough to train on a single small GPU.
IN PLAIN WORDS	A cheap, memory-light way to teach the model. Instead of rewriting its whole brain, we attach small add-ons it learns on — light enough to train on a free GPU.

Frozen base, loaded in 4-bit	
TECHNICAL	The base model's weights are held fixed (“frozen”) and stored at 4-bit precision during training. Gradients flow only into the adapters, so the large model is never updated and uses roughly 4× less memory.
IN PLAIN WORDS	We lock the original brain so it can't be damaged, and store it in a compressed form to save memory. Only the add-ons change.

Lightweight adapters (< 1% of parameters)

TECHNICAL	The trainable LoRA matrices hold under 1% of the model's total parameters; everything else stays frozen. Fast to train and tiny to store and share.
IN PLAIN WORDS	We only train a very small add-on — fewer than 1 in 100 of the model's dials — so training is quick and the result is a small file.

Rank 16 (LoRA rank, r)

TECHNICAL	The inner dimension of the low-rank adapter matrices. Higher rank means more learning capacity but more parameters; 16 balances capacity against overfitting on a small dataset.
IN PLAIN WORDS	How “big” the add-on is allowed to be. 16 is a middle setting — enough to learn our style without memorising the small dataset.

Learning rate 1e-4

TECHNICAL	The step size for gradient updates. 1e-4 (0.0001) is deliberately small, making gentle updates that reduce overfitting on limited data.
IN PLAIN WORDS	How big a change the model makes at each step. We keep it small so it learns gradually instead of over-reacting.

2 epochs

TECHNICAL	The training set is passed through twice. Using few epochs limits overfitting on a small dataset.
IN PLAIN WORDS	The model reads the whole study set two times — enough to learn the style, not so much that it just memorises answers.

507 Q&A; pairs (483 grounded + 24 safety / refusal)

TECHNICAL	The supervised fine-tuning dataset: 483 grounded parenting question-answer pairs, plus 24 pairs that teach safe refusals (emergencies, dosing, myths, out-of-scope).
IN PLAIN WORDS	507 example questions-and-answers we taught it with — 483 normal parenting ones, and 24 that teach it when to say “see a doctor” instead of answering.

Train / Validation / Held-out test (482 / 25 / 28, zero overlap)

TECHNICAL	A standard split: 482 pairs to train on, 25 to validate and pick the best checkpoint, and a 28-item held-out test with no overlap, used for an unbiased final score.
IN PLAIN WORDS	We split the examples so the model studies on some, we check its progress on others, and we grade it on 28 questions it has never seen — so the score is honest.

“Tune for tone and safety, not facts”

TECHNICAL Fine-tuning targets style, structure, and refusal behaviour. Factual grounding is delegated to RAG at answer time, so facts (which go stale or get hallucinated) are not baked into the weights.

IN PLAIN WORDS We teach it how to talk and when to be careful — not the facts themselves. The facts come from trusted sources at the moment it answers.

SLIDE 9 · OPTIMIZATION (QUANTIZATION)

Small enough to run on a laptop

“Post-training quantization to GGUF Q4_K_M takes the model from about 8 GB down to 2.5 — roughly three times smaller — and it runs on a laptop CPU at about 9 tokens per second, with no GPU required to serve. Same behaviour, a third of the size, so it runs right next to the RAG stack, anywhere.”

Post-training quantization

TECHNICAL Reducing the numerical precision of a trained model's weights (for example 16-bit → 4-bit) after training. This shrinks size and speeds up CPU inference with minimal quality loss.

IN PLAIN WORDS After training, we compress the model — storing its numbers more roughly — so it becomes smaller and faster, with barely any drop in quality.

GGUF

TECHNICAL A single-file model format used by llama.cpp for efficient CPU and edge inference. It packages the weights plus metadata for portable local hosting.

IN PLAIN WORDS A tidy, single-file package of the model that is easy to run on a normal computer.

Q4_K_M

TECHNICAL A specific 4-bit “K-quant, medium” scheme that balances file size against accuracy — a widely used sweet spot for quality versus footprint.

IN PLAIN WORDS The exact “compress to 4-bit, medium quality” setting we chose: small, but still good.

~8 GB → ~2.5 GB (~3× smaller)

TECHNICAL Quantization cuts the on-disk and in-memory footprint from about 8 GB (16-bit) to about 2.5 GB — roughly a 3× reduction.

IN PLAIN WORDS The model shrank from about 8 GB to 2.5 GB — about a third of the size — so it fits on ordinary machines.

Laptop CPU · ~9 tokens / second

TECHNICAL Inference throughput of ~9.2 tokens per second under greedy decoding on a laptop CPU — usable interactive speed with no GPU.

IN PLAIN WORDS On a normal laptop (no graphics card), it produces about 9 words-worth of text per second — fast enough to hold a conversation.

No GPU required to serve

TECHNICAL The quantized model runs via llama.cpp / Ollama on CPU, removing the GPU dependency at serving time and lowering deployment cost.

IN PLAIN WORDS You don't need expensive special hardware to run it — a regular computer is enough.

“Runs right next to the RAG stack”

TECHNICAL The quantized model is the RAG layer's default generator (nile_chat_gguf), co-located with retrieval so the whole pipeline can run locally on one machine.

IN PLAIN WORDS It can sit right beside the search system, so the entire assistant runs together in one place.